Partie 1
Les pointeurs

INFO410 – Structures de données classiques

1. Variables statiques

- Jusque là les variables étaient **statiques**
  - Leur emplacement mémoire
  - Leur taille

  étaient fixés à la **compilation**

- Cela pose problème lorsque l'on ne connait pas à l'avance le nombre d'éléments que l'on va manipuler dans un tableau par exemple

- Nécessité de désigner des **emplacements mémoires** spécifiques

1. Variables statiques

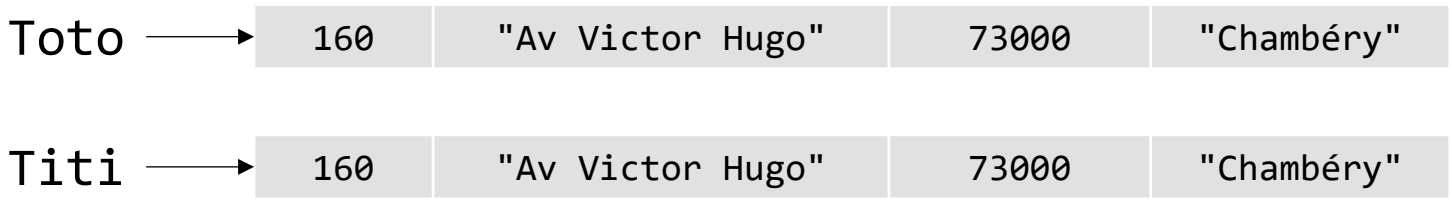
- Exemple avec des enregistrement de type habitation,

```
type Domicile is record
   numero: integer;
   rue: Unbounded_String;
   code_postal: integer;
   commune: Unbounded_String;
end record;
```

1. Variables statiques

- Toto et Titi habitent à la même adresse,
  on souhaite donc créer un 2e moyen d'accéder à une même variable

```
Toto, Titi: Domicile;
Toto := (160, "Av Victor Hugo", 73000, "Chambéry");
Titi := Toto;
```

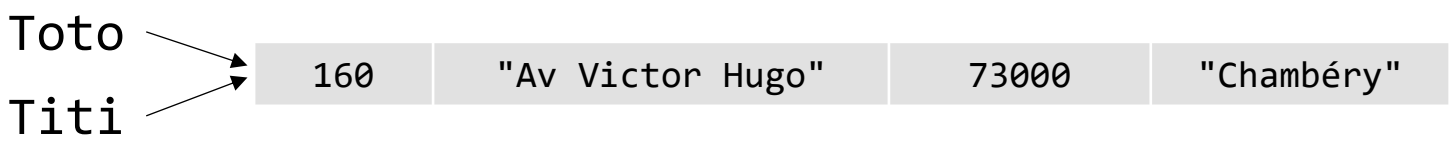


1. Variables statiques

- Problème : si on modifie toto, titi ne sera pas modifiée…
  Les deux variables désignent des emplacements mémoires différents

```
Toto.numero := 4;
put(Titi.numero);
```

- Solution avoir un nouveau type de variable qui contiennent les emplacements mémoires : les pointeurs



2. Variables dynamiques

- Les pointeurs contiennent donc des valeurs particulières

- Ils permettent de créer (ou supprimer) de **nouvelles variables** (non déclarées) en fonction des besoins pendant **l'exécution** du programme

- Ces variables sont dites **dynamiques** : leur emplacement mémoire est **alloué** par une **instruction** spéciale et non par **déclaration** (comme avec les variables statiques)

2. Variables dynamiques

- La notation suivante est utilisée pour introduire le type pointeur :

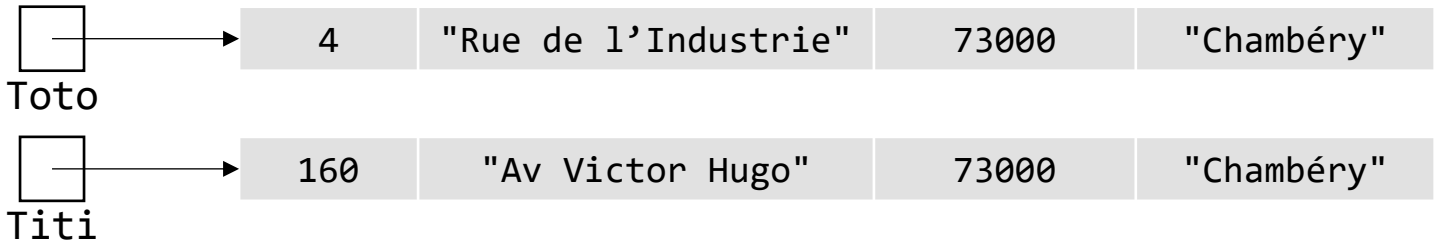
```
type T_pointeur is access T;
```

- On peut dire qu'une variable de type `T_pointeur` est un pointeur vers une donnée de type `T`

```
type Pt_Domicile is access Domicile;
Toto, Titi: Pt_Domicile;
```

2. Variables dynamiques

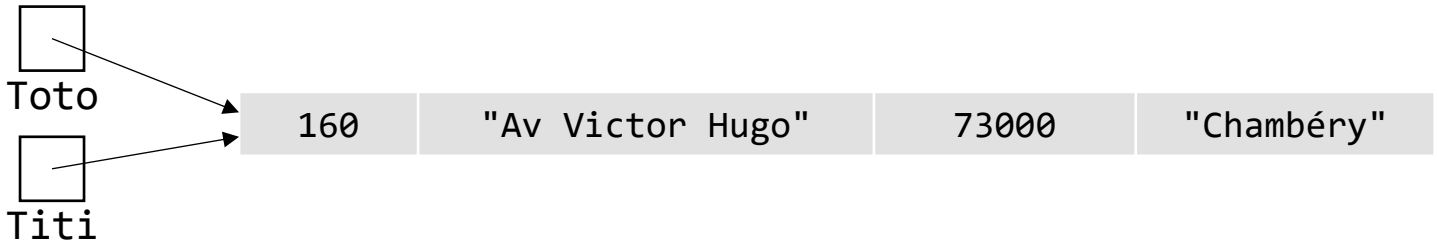
```
Toto := new Domicile;
Toto.numero := 4;
Toto.rue := "Rue de l’Industrie";
…
Titi := new Domicile;
Titi.rue := 160;
…
```



2. Variables dynamiques

- `new Domicile` crée une variable et retourne son **emplacement**

```
Toto := new Domicile;
Toto.numero := 160;
Toto.rue := "Av Victor Hugo";
…
Titi := Toto;
put(Titi.numero);
…
```



3. Utilisation des pointeurs

- Encore des exemples,

```
type Personne is record
   nom, tel: Unbounded_String;
end record;
type Pt_Personne is access Personne;
```

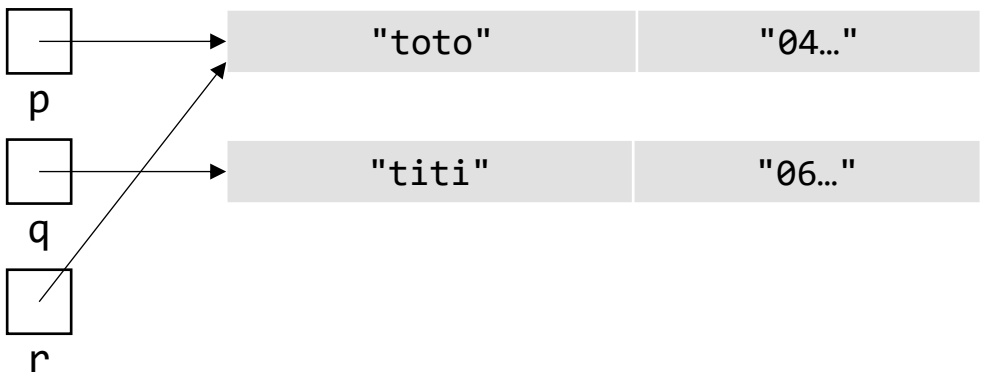
- `p`, `q` et `r` pointent vers des variables dynamiques de type `Personne`

```
p := new Personne;
q := new Personne;
```

3. Utilisation des pointeurs

- Création de 2 variables pointées p et q qui contiennent leur adresse
- La perte du contenu d'un pointeur entraine la perte de l'information vers laquelle il pointe

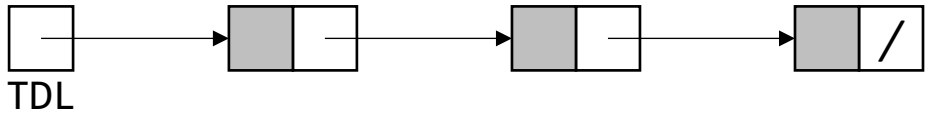
```
p.nom := "toto";
q.nom := "titi";
q.tel := "06…";
r := p;
r.tel := "04…";
```



4. Listes chainées

- Les variables dynamiques permettent une implémentation de certaines structures de données : listes, piles, arbres, etc.

- Ces structures sont caractérisées par le fait que leurs éléments possèdent entre eux des liens représentés par des pointeurs



4. Listes chainées

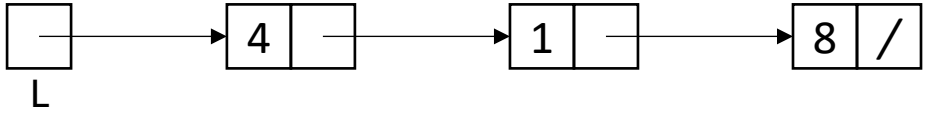
- Une liste chainée est une suite de nœuds liés entre eux
- Chaque nœud contient son information et un pointeur vers le suivant

```
type noeud;
type liste is access noeud;
type noeud is record
   info: integer;
   suiv: liste;
end record;
```

4. Listes chainées

- Affichage d’une liste chainée

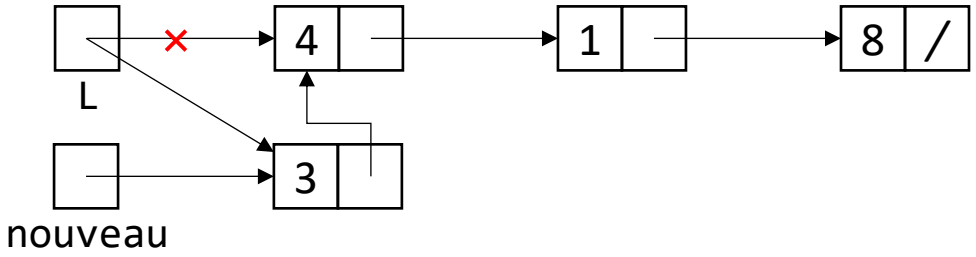
```
procedure afficher(L: in liste) is
    …
end afficher;
```



4. Listes chainées

- Insertion d'un élément dans une liste chainée (en tête de liste)

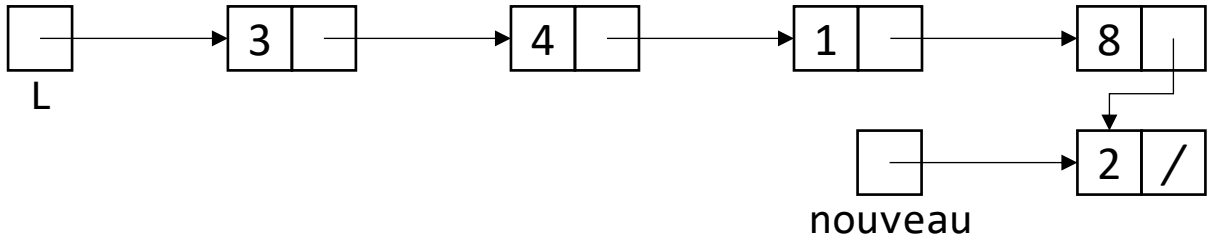
```
procedure inserer_en_tete(
                        L: in out liste; elem: in integer) is
   …
end inserer_en_tete;
```



4. Listes chainées

- Insertion d'un élément dans une liste chainée (en fin de liste)

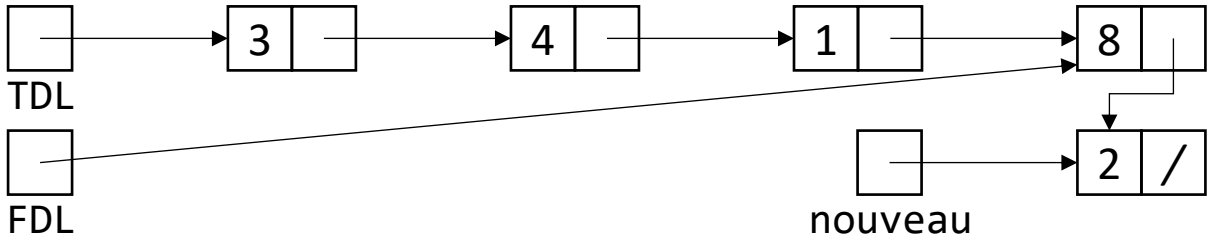
```
procedure inserer_en_fin(
                        L: in out liste; elem: in integer) is
   …
end inserer_en_fin;
```



4. Listes chainées

- Insertion d'un élément dans une liste chainée (en fin de liste)

```
procedure inserer_en_fin(
                TDL, FDL: in out liste; elem: in integer) is
   …
end inserer_en_fin;
```

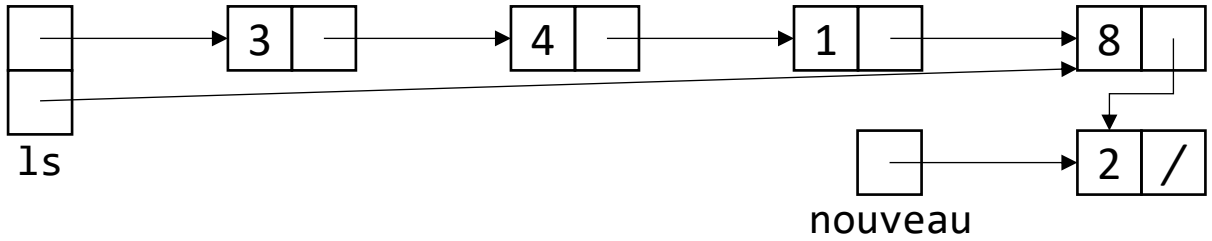


4. Listes chainées

- Insertion d'un élément dans une liste sioux (en fin de liste)

```
type liste_sioux is record
   tdl, fdl: liste;
end record;

procedure inserer_en_fin(
                LS: in out liste_sioux; elem: in integer) is
   …
```



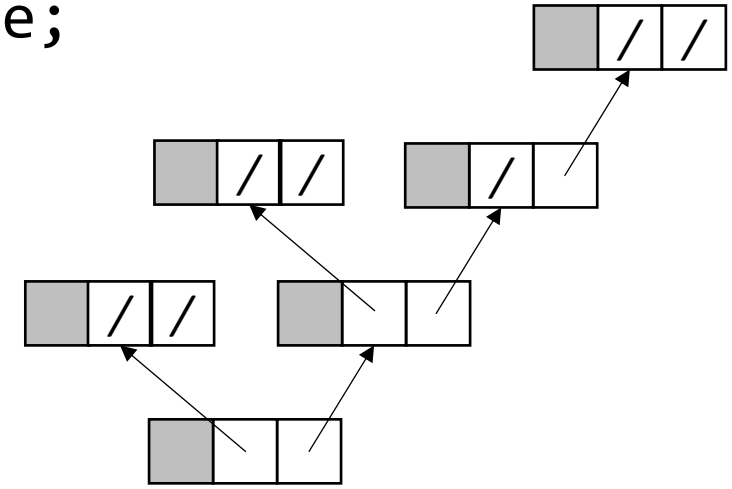
4. Listes chainées

- Création d’une liste d’entiers (terminée par -1) entrés par l’utilisateur et affichés dans l’ordre inverse

```
procedure liste_a_l_envers is
   …
end liste_a_l_envers;
```

5. Arbres généalogiques

```
type personne;
type ancetre is access personne;
type personne is record
prenom: string(1..20);
   pere, mere: ancetre;
end record;
```



5. Arbres généalogiques

- Affichage de tous les ancêtres d'une personne

```
procedure affiche_ancetres(A: ancetre) is
   …
end affiche_ancetres;
```